

# Лекція 16



# NoSQL



# Способи зберігання даних в NoSQL

## Key-Value

- DynamoDB
- Riak
- Redis
- MemcacheDB

## Column Store

- Hadoop /  
HBase
- Cassandra;

## Document Store

- MongoDB
- ElasticSearch
- CouchDB

## Graph DBMS

- Neo4J
- Infinite Graph
- Sparksee

## Multimodel DBMS

- ArangoDB
- OrientDB
- Oracle NOSQL  
Database

## Object DBMS

- Versant
- db4o
- Objectivity
- ObjectDB

<https://hostingdata.co.uk/nosql-database/>

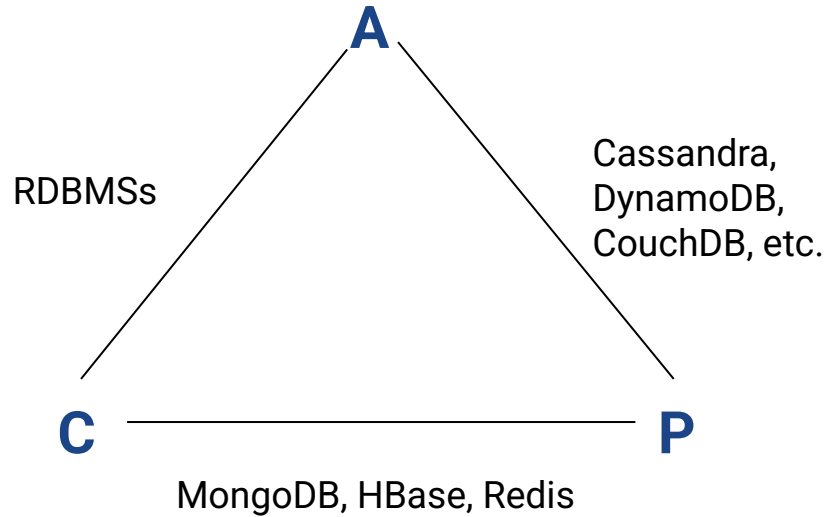
# DB-Engines Ranking (по популярності)

420 systems in ranking, May 2024

| Rank     |          |          | DBMS                         | Database Model               | Score    |          |          |
|----------|----------|----------|------------------------------|------------------------------|----------|----------|----------|
| May 2024 | Apr 2024 | May 2023 |                              |                              | May 2024 | Apr 2024 | May 2023 |
| 1.       | 1.       | 1.       | Oracle +                     | Relational, Multi-model ⓘ    | 1236.29  | +2.02    | +3.66    |
| 2.       | 2.       | 2.       | MySQL +                      | Relational, Multi-model ⓘ    | 1083.74  | -3.99    | -88.72   |
| 3.       | 3.       | 3.       | Microsoft SQL Server +       | Relational, Multi-model ⓘ    | 824.29   | -5.50    | -95.80   |
| 4.       | 4.       | 4.       | PostgreSQL +                 | Relational, Multi-model ⓘ    | 645.54   | +0.49    | +27.64   |
| 5.       | 5.       | 5.       | MongoDB +                    | Document, Multi-model ⓘ      | 421.65   | -2.31    | -14.96   |
| 6.       | 6.       | 6.       | Redis +                      | Key-value, Multi-model ⓘ     | 157.80   | +1.36    | -10.33   |
| 7.       | 7.       | ↑ 8.     | Elasticsearch                | Search engine, Multi-model ⓘ | 135.35   | +0.57    | -6.28    |
| 8.       | 8.       | ↓ 7.     | IBM Db2                      | Relational, Multi-model ⓘ    | 128.46   | +0.97    | -14.56   |
| 9.       | 9.       | ↑ 11.    | Snowflake +                  | Relational                   | 121.33   | -1.87    | +9.61    |
| 10.      | 10.      | ↓ 9.     | SQLite +                     | Relational                   | 114.32   | -1.69    | -19.54   |
| 11.      | 11.      | ↓ 10.    | Microsoft Access             | Relational                   | 104.92   | -0.49    | -26.26   |
| 12.      | 12.      | 12.      | Cassandra +                  | Wide column, Multi-model ⓘ   | 101.89   | -1.97    | -9.25    |
| 13.      | 13.      | 13.      | MariaDB +                    | Relational, Multi-model ⓘ    | 93.21    | -0.60    | -3.66    |
| 14.      | 14.      | 14.      | Splunk                       | Search engine                | 86.45    | -2.26    | -0.19    |
| 15.      | ↑ 17.    | ↑ 18.    | Databricks +                 | Multi-model ⓘ                | 78.61    | +2.28    | +14.66   |
| 16.      | ↓ 15.    | 16.      | Microsoft Azure SQL Database | Relational, Multi-model ⓘ    | 77.99    | -0.41    | -1.21    |

<https://db-engines.com/en/ranking>

# CAP-теорема



**C (consistency)** — узгодженість. Усі вузли бачать однакові дані у будь-який момент часу.

**A (availability)** — доступність. Гарантія того, що кожен запит отримає коректну відповідь.

**P (partition tolerance)** — устійчивість до розділення. Попри розділення на ізольовані секції або втрати зв'язку з частиною вузлів, система не втрачає стабільність і здатність коректно відповідати на запити.

# BASE instead of ACID

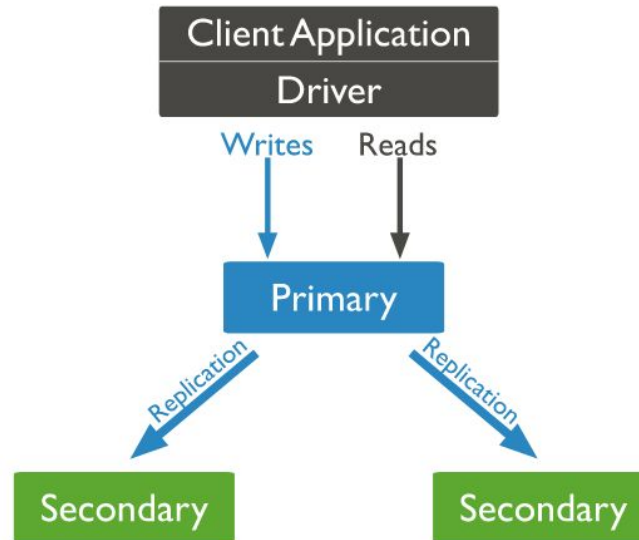
- **Basic Availability.** Система відповідає на будь-який запит, але ця відповідь може мати помилку або неузгоджені дані.
- **Soft-state.** Стан системи може змінюватися з часом через зміни узгодженості в кінцевому рахунку.
- **Eventual consistency** (узгодженості в кінцевому рахунку). Система колись стане узгодженою. Вона буде продовжувати приймати дані і не буде перевіряти кожну транзакцію на узгодженість.

# MongoDB features

- Document-Oriented Storage
- Open source
  - there's an OpenSource edition
- Written in C++
- Full Index Support
- Replication & High Availability
- Fast In-Place Updates
- Map/Reduce
- GridFS
- Transactions

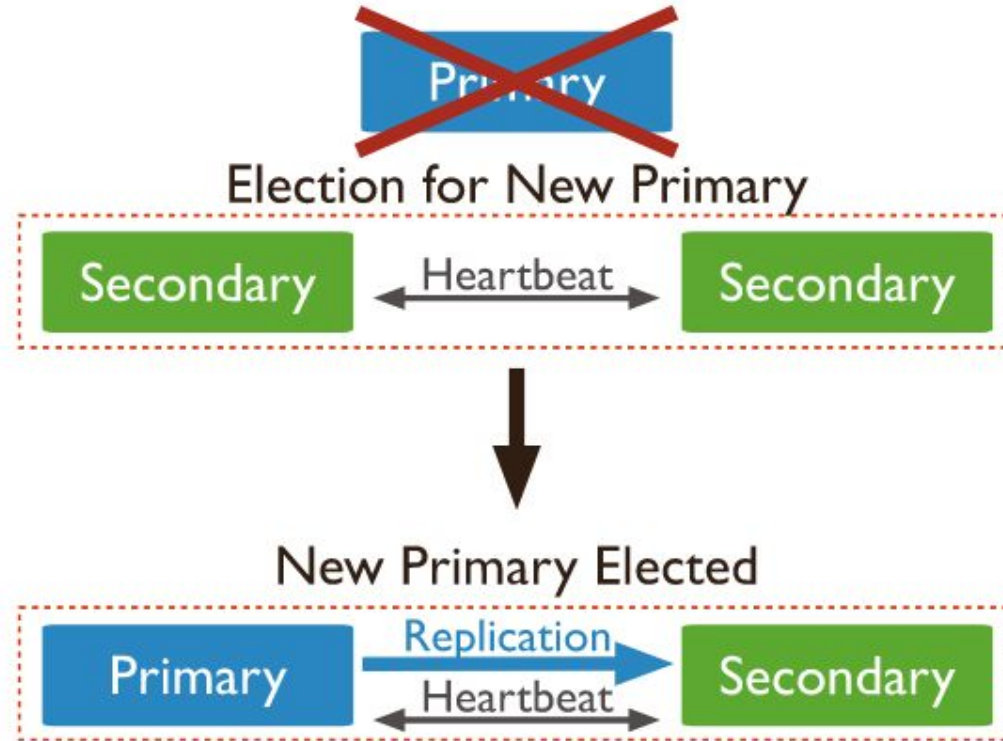
# Replica set in MongoDB

A **Replica set** in MongoDB is a group of mongod processes that maintain the same data set. Replica sets provide redundancy and high availability, and are the basis for all production deployments.

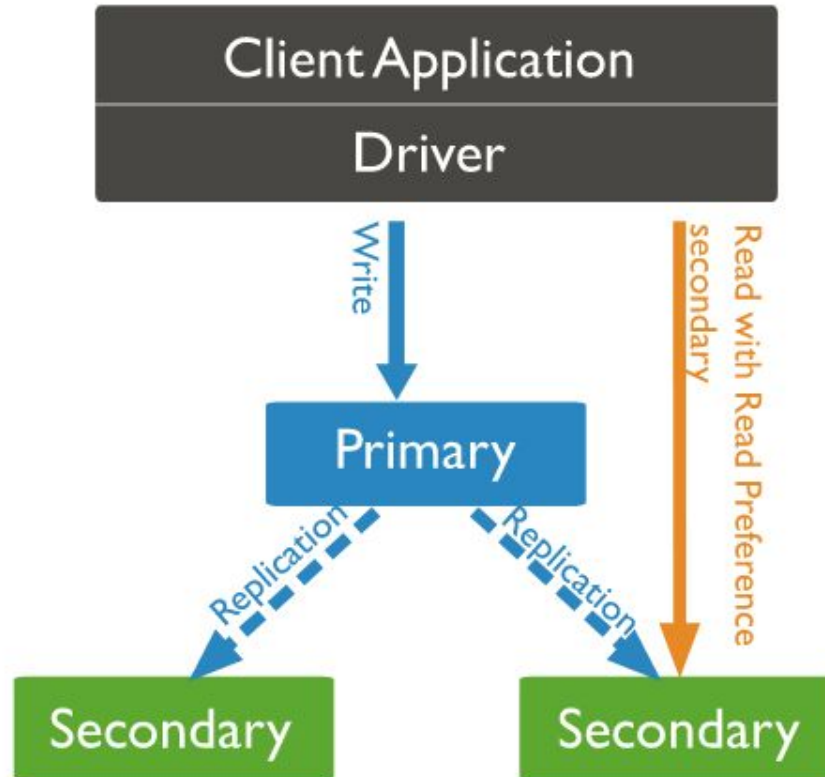




# Automatic failover



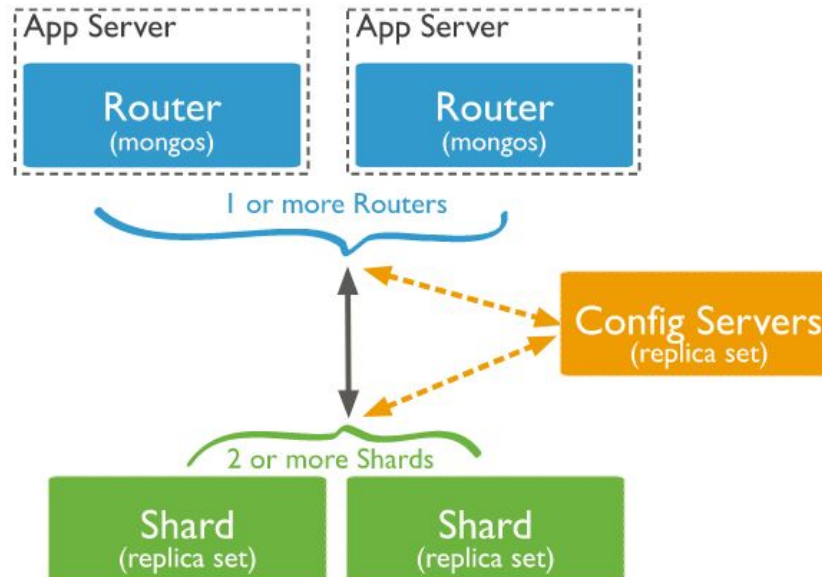
# Read Preference



# Sharding

**Sharding** - это способ распределения данных на нескольких серверах

MongoDB использует **shard key** для распределения документов коллекции по shards



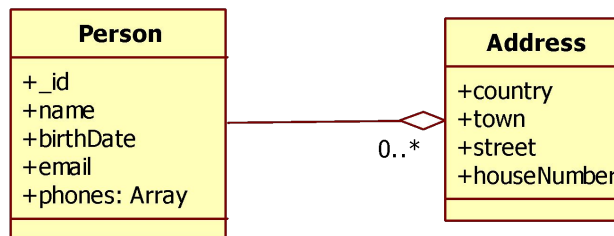
# Модель зберігання даних

- База даних складається з колекцій
- В колекціях зберігаються документи
- Документ має пари ключ-значення
- Значеннями можуть бути
  - Значення простих типів (строка, число, дата, boolean, etc.)
  - Масиви
  - Вложені документи
- Максимальний розмір документу – 16 MB
  - Для зберігання даних більшого розміру можна використовувати GridFS

# Приклад документу в форматі JSON

```
{  
  "_id" : ObjectId("51ca9b6f780d70976184f2ca"),  
  "name" : "Taras",  
  "surname" : "Shevchenko",  
  "groupId" : new NumberLong(1001),  
  "category" : "VIP",  
  "birthDate" : ISODate("1814-03-09T00:00:00Z"),  
  "email" : "shevchenko@pochti.net"  
}
```

# Вкладені документи і масиви



```
{
  "_id" : ObjectId("51ca9b6f780d70976184f2ca"),
  "surname" : "Shevchenko",
  "phones" : ["+012345678", "+987654321"], // масив простих типів
  "addresses" : [                          // масив вкладених документів
    {
      "country" : "UA",
      "town" : "Moryntsi",
    },
    {
      "country" : "UA",
      "town" : "Kiev",
      "street": "..."
    }
  ]
}
```

# Встановлення MongoDB

<https://www.mongodb.com/try/download/community>

Version

7.0.9 (current)



Platform

Windows x64



Package

msi



Download



Copy link

More Options



# Або запуск в Docker

Файл *docker-compose.yml*

```
version: '3.1'
```

```
services:
```

```
  mongo:
```

```
    image: mongo:7.0.9
```

```
    container_name: 'mongo-sample-db'
```

```
    ports:
```

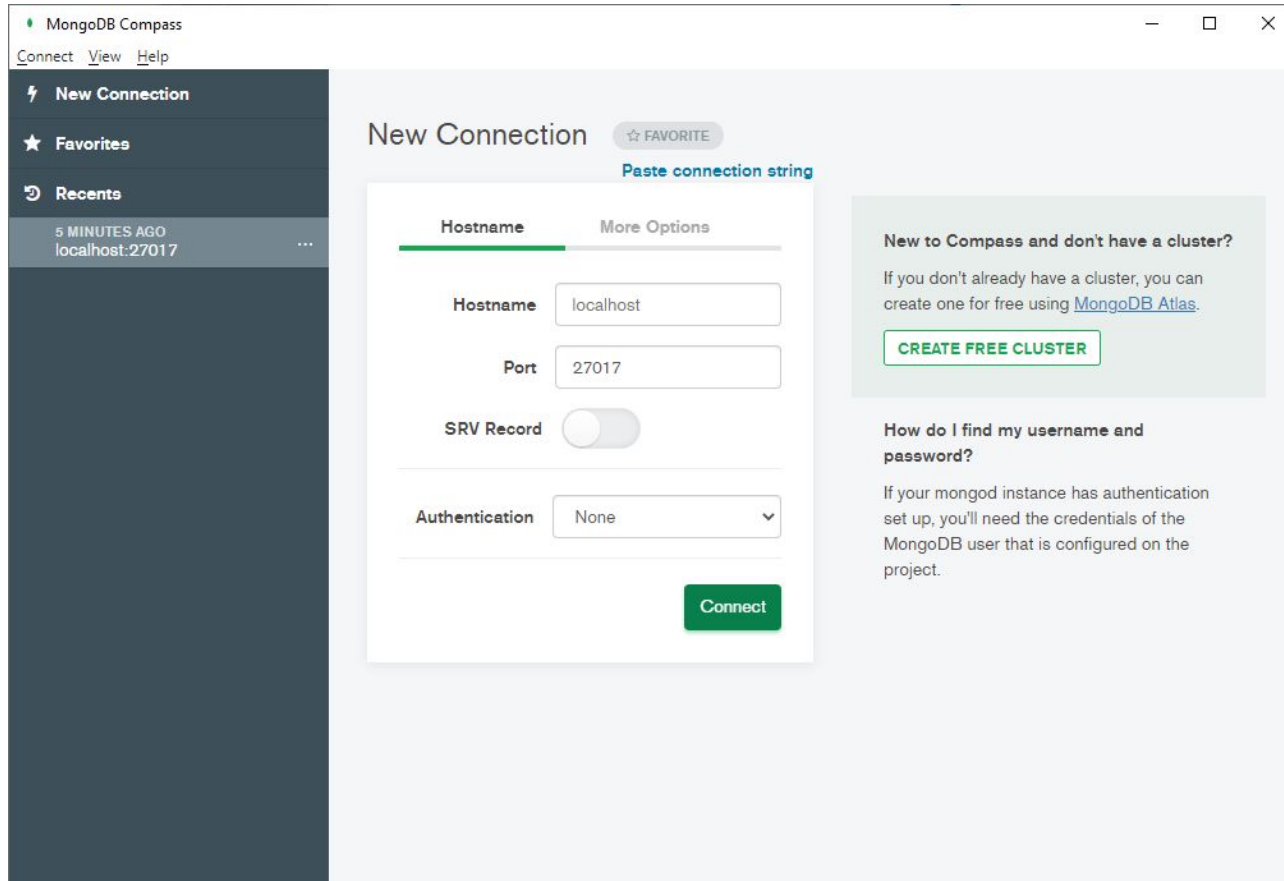
```
      - "27017:27017"
```

## Запуск

```
docker-compose up
```



# MongoDB Compass



The screenshot shows the MongoDB Compass application window. On the left is a dark sidebar with a menu containing 'New Connection' (with a lightning bolt icon), 'Favorites' (with a star icon), and 'Recents' (with a circular arrow icon). Under 'Recents', there is an entry '5 MINUTES AGO localhost:27017' with a three-dot menu icon to its right. The main area of the window is titled 'New Connection' and has a '☆ FAVORITE' button. Below the title is a link 'Paste connection string'. The form is divided into two tabs: 'Hostname' (which is selected and underlined in green) and 'More Options'. In the 'Hostname' tab, there are three input fields: 'Hostname' with the value 'localhost', 'Port' with the value '27017', and 'SRV Record' which is a toggle switch currently turned off. Below these is an 'Authentication' dropdown menu set to 'None'. A green 'Connect' button is at the bottom right of the form. To the right of the form is a light green informational box with the heading 'New to Compass and don't have a cluster?'. It contains the text 'If you don't already have a cluster, you can create one for free using [MongoDB Atlas](#).' and a green button labeled 'CREATE FREE CLUSTER'. Below this box is another section titled 'How do I find my username and password?' with the text 'If your mongod instance has authentication set up, you'll need the credentials of the MongoDB user that is configured on the project.'

MongoDB Compass

Connect View Help

**New Connection** ☆ FAVORITE

Paste connection string

**Hostname** More Options

Hostname localhost

Port 27017

SRV Record ☐

Authentication None

Connect

**New to Compass and don't have a cluster?**

If you don't already have a cluster, you can create one for free using [MongoDB Atlas](#).

**CREATE FREE CLUSTER**

**How do I find my username and password?**

If your mongod instance has authentication set up, you'll need the credentials of the MongoDB user that is configured on the project.

# Mongo Console

mongosh

a6o

docker exec -it mongo-sample-db mongosh

```
mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+1.6.2
PS C:\Users\Andrii> docker exec -it mongo-sample-db mongosh
Current Mongosh Log ID: 63eb5eca0e888c40a5101647
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+1.6.2
Using MongoDB:      5.0.14
Using Mongosh:      1.6.2

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2023-02-14T10:09:05.821+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http
://dochub.mongodb.org/core/prodnotes-filesystem
2023-02-14T10:09:06.382+00:00: Access control is not enabled for the database. Read and write access to data and configurati
on is unrestricted
2023-02-14T10:09:06.382+00:00: /sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never'
-----

-----
Enable MongoDB's free cloud-based monitoring service, which will then receive and display
metrics about your deployment (disk utilization, CPU, operation statistics, etc).

The monitoring data will be available on a MongoDB website with a unique URL accessible to you
and anyone you share the URL with. MongoDB may use this information to make product
improvements and to suggest MongoDB products and deployment options to you.

To enable free monitoring, run the following command: db.enableFreeMonitoring()
To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
-----

test> |
```

# Робота в консолі MongoDB

```
// перелік всіх баз даних  
show databases;
```

```
// вибрати базу даних  
use <database_name>;
```

```
// подивитись всі колекції в вибраній базі даних  
show collections;
```

# Вставка и обновление

*// ВСТАВКА*

```
db.students.insertOne({  
  "name" : "Taras",  
  "email" : "taras.shevchenko@gmail.com",  
  "birthDate": new ISODate('1814-03-09')  
});
```

*// ОНОВЛЕННЯ*

```
db.students.updateMany(  
  // критерій  
  {"name" : "Taras"},  
  // встановлює значення полів  
  {$set : {"category" : "VIP"}}  
);
```

# Write concerns

**Write concern** describes the **level of acknowledgment** requested from MongoDB for write operations to a standalone **mongod** or to replica sets or to sharded clusters.

```
db.students.insertOne({
  "name" : "Ivan",
  "email" : "ivan.franko@gmail.com"
},
{
  "writeConcern": {
    "w": "majority",      // на скільки нод очікувати запис
    "j": true,           // очікувати запис в oplog на диск
    "wtimeout": 5000     // таймаут очікування в ms
  }
});
```

# Вибірка даних

*// вибірка всіх записів колекції*

```
db.students.find();
```

*// вибірка 5-ти елементів, починаючи з 11-го*

```
db.students.find().skip(10).limit(5);
```

*// вибірка студентів з ім'ям і датою народження після*

```
db.students.find({  
    "name" : "Taras",  
    "birthDate" : {$gte : new ISODate('1800-01-01')}  
});
```

# Операція findAndModify

```
// робить update и повертає документ, що був в БД до оновлення  
db.authenticationToken.findAndModify({  
  query: {"userId" : 123123},  
  update : {  
    $set : {expirationTime : new Date().getMilliseconds() +  
1080000000},  
    $inc : {count : 1}    // збільшити значення на 1  
  },  
  upsert : true,    // робить вставку, якщо не знайшли  
});
```

```
// якщо хочемо побачити документ, яким він став після оновлення,  
// то треба вказати  
new : true
```

# Aggregation Framework

```
db.students.aggregate([  
    // задає критерій вибірки  
    $match: {  
        "groupId" : {$ne : null},  
    },  
    {  
        // групуємо по ID груп  
        $group: {  
            _id : {groupId: "$groupId"},  
            "count" : {$sum : 1}  
        },  
    },  
    {  
        // сортуємо результат по зменшенню кількості  
        $sort: {  
            "count" : -1  
        }  
    }  
])
```



# Views

```
db.createView(  
    "studentsStat",    // назва view  
    "students",        // назва колекції  
    [  
        { $match: { "groupId" : {$ne : null} } },  
        { $group: {  
            _id : {groupId : "$groupId"},  
            "count" : {$sum : 1}  
        }},  
        { $sort: {"count" : -1} }  
    ]  
);
```

*// так потім робимо запити до view*

```
db.studentsStat.find();
```

# Aggregation with \$lookup (aka JOIN)

```
// йдемо по всіх групах
db.groups.aggregate( [{
  // під'єднуємо студентів
  $lookup: {
    from: "students",
    localField: "_id",
    foreignField: "groupId",
    as: "students"
  }
}, {
  // залишаємо тільки ті поля які нам потрібні і
  // рахуємо студентів
  $project: {
    _id: 1,
    name: 1,
    studentsCount: { $size: "$students" }
  }
}]);
```

# Indexes

*// індекс по одному полю*

```
db.students.createIndex({"groupId" : 1});
```

*// індекс по кількох полях*

```
db.students.createIndex(  
  {"surname" : 1, "name" : 1},  
  { name: "index by name and surname"  
});
```

*// повертає список всіх індексів в колекціях*

```
db.students.getIndexes();
```

*// перевірити використання індексів в запиті*

```
db.students.find({...}).explain()
```

# Налаштування індексів

*// унікальний індекс*

```
db.students.createIndex(  
  {"email" : 1},  
  {unique:true}  
);
```

*// не індексує документи, якщо поле пусте*

```
db.students.createIndex({"birthDate":1}, {sparse:true});
```

*// TTL індекс - автоматично видаляє дані в певний час*

```
db.events_log.createIndex(  
  {"createdAt": 1},  
  {expireAfterSeconds: 86430}  
);
```

Використовується для збереження файлів, розмір яких может бути в т.ч. більше 16 МБ

- Файли розбиваються на секції (chunks), кожна з яких зберігається в окремому документі в спеціальній колекції
- Метадані файлу (назва, розмір, і т.і.) зберігаються в окремій колекції

## Переваги GridFS

- Метадані файлів зберігаються цілісно з їх вмістом
- Доступність через connection, як і до БД
- Реплікація
- Можливість доступу до окремих блоків файлів без викачування всього вмісту

# Transactions

В MongoDB операція над одним документом - атомарна

- За рахунок використання вкладених документів і масивів, модифікація структур даних, що є частиною одного документу, є “транзакційною”

З версії 4.0, MongoDB підтримує multi-document transactions

- Коли вони потрібні, їх потрібно відкривати і закривати ЯВНО

<https://docs.mongodb.com/manual/core/transactions/>

# Spring Data MongoDB

- Легка конфігурація підключення до MongoDB
- Маппінг за допомогою анотацій
- Спрощене написання запитів і конвертацій за допомогою MongoTemplate, Query, Criteria, Update DSLs та MongoRepository
- Підтримка GridFS

# Dependencies (pom.xml)

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.0.2</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>

...

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
  </dependency>
  ...
</dependencies>
```



# Малпінг Data-об'єктів

@Getter

@Setter

@Document("students")

public class StudentData {

@Id

private String id;

private String name;

private String groupId;

private LocalDate birthDate;

private List<String> phoneNumbers;

private Address address;

}

# MongoRepository

Інтерфейс `MongoRepository` має багато методів для роботи з сутністю:

- `save`
- `findById`
- `delete`
- `etc.`

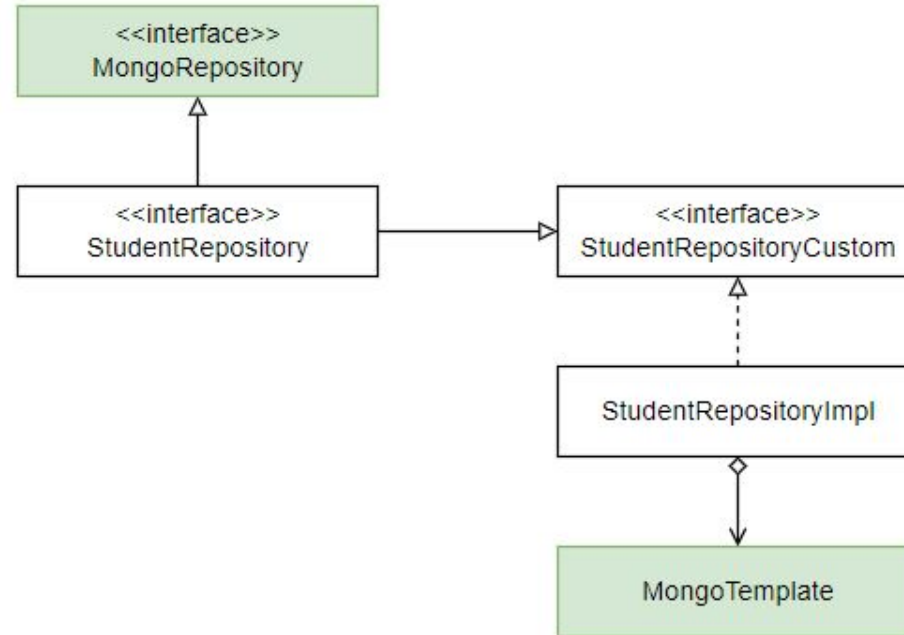
```
public interface StudentRepository extends  
    MongoRepository<StudentData, String> {  
  
    // запит буде формуватися автоматично за назвою метода  
    Stream<StudentData> findByGroupId(String groupId);  
  
}
```

# MongoTemplate

Дозволяє виконувати довільні запити

```
public List<StudentData> search(StudentQueryDto query) {  
    Query mongoQuery = new Query();  
    if (StringUtils.isNotBlank(query.getName())) {  
        mongoQuery.addCriteria(where(StudentData.Fields.name)  
                                .is(query.getName()));  
    }  
    if (StringUtils.isNotBlank(query.getGroupId())) {  
        mongoQuery.addCriteria(where(StudentData.Fields.groupId)  
                                .is(query.getGroupId()));  
    }  
    return mongoTemplate.find(mongoQuery, StudentData.class);  
}
```

# Поеднуємо MongoRepository і MongoTemplate



# application.properties

spring.data.mongodb.uri: **mongodb://localhost:27017/students**

# Integration Tests: Embedded MongoDB

In pom.xml:

```
<dependency>  
  <groupId>de.flapdoodle.embed</groupId>  
  <artifactId>de.flapdoodle.embed.mongo.spring30x</artifactId>  
  <version>4.5.1</version>  
  <scope>test</scope>  
</dependency>
```

In test/resources/application.properties:

```
de.flapdoodle.mongodb.embedded.version=5.0.14
```

# Integration Tests

```
/**
 * Tests REST endpoints /students/* provided by {@link StudentController}.
 */
@SpringBootTest(
    webEnvironment = SpringBootTest.WebEnvironment.MOCK,
    classes = MongoSampleApplication.class)
@AutoConfigureMockMvc
class StudentControllerTest {

    @Autowired
    MockMvc mvc;

    @Autowired
    StudentRepository studentRepository;

    ...
}
```

# Integration Test case

@Test

```
void testCreateStudent_success() throws Exception {  
    String groupId = groupRepository.save(buildGroup("Group1")).getId();  
    String name = "Taras";  
    String surname = "Shevchenko";  
    String body = ""  
        {  
            "name": "%s",  
            "surname": "%s",  
            "groupId": "%s"  
        }  
        """.formatted(name, surname, groupId);  
    MockHttpServletResponse response = mvc.perform(post("/api/students")  
        .contentType(MediaType.APPLICATION_JSON)  
        .content(body))  
        .andExpect(status().isCreated())  
        .andReturn().getResponse();  
  
    RestResponse result = TestUtil.parseJson(response.getContentAsString(), RestResponse.class);  
    String studentId = result.getResult();  
    assertThat(studentId).isNotEmpty();  
    Optional<StudentData> student = studentRepository.findById(studentId);  
    assertThat(student).isPresent();  
    assertThat(student.get().getName()).isEqualTo(name);}
```





# Projections and Aggregations in Spring Data

<https://www.baeldung.com/spring-data-mongodb-projections-aggregations>

# MongoDB University

MongoDB University

All Learning

Certifications ▾

## Featured Training

See All Content →



### MongoDB Developer Learning Paths

These learning paths combine a series of courses to teach you MongoDB skills with your programming language of choice.



### Introduction to MongoDB

You'll be guided through the foundational skills and knowledge you need to get started with MongoDB.



### Node.js Developer Path

In this path, you'll learn the basics of building modern applications with Node.js, using MongoDB as your database.



### Associate Developer Exam

When you become MongoDB Certified, you're achieving the same level of expertise that we require of our own developers.

# 100%

Free Content Access

# 24/7

Certification Access

# 150+

Labs

<https://learn.mongodb.com/>